

Personal Expense Tracker

A Modular Python Application with Excel Storage

PROJECT REPORT

Name	Sonam Tshering Gyeltshen
Registration Number	26BAS10105
Course Name & Code	CSE - 1021
Faculty / Instructor	Dr. Anantha Kumaran S
Submission Date	29/09/2026

Table of Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams (Use Case, Workflow, Sequence, Class/Component, ER)
8. Design Decisions & Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings & Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Managing money starts with knowing where it goes. If a person does not keep track of daily spending, it becomes difficult to stay within a budget or save money. The Personal Expense Tracker is a simple menu-driven Python application that helps users record their expenses, organise them into categories, set a budget, and view spending summaries. All the information is stored in an Excel file (expenses.xlsx), so the program works completely offline and the data can be opened in Microsoft Excel at any time.

This project uses basic Python programming concepts such as modular programming, file handling, CRUD operations, input validation, exception handling, logging, and automated testing. The program is divided into nine Python modules and a test package, making the code easier to understand, test, and maintain.

Project at a glance

Item	Detail
Language	Python 3.9+
Interface	Command-line menu (11 options)
Storage	Excel workbook via openpyxl - sheets: Expenses, Categories, Budget
Modules	main, expense, categories, budget, report, database, validators, logger, utils (9 source files)
Tests	18 unit tests using unittest
Version control	Git / GitHub (.gitignore excludes data, logs and caches)

3. Problem Statement

Students and individuals often find it difficult to keep track of their daily spending. They may remember expenses roughly or write them down in different places, making it hard to see where their money is going. It can also be difficult to quickly check whether their spending is within their monthly budget. Using excel spreadsheets can lead to mistakes such as negative or incorrect amounts, missing categories, or duplicate entries.

To solve this problem, a simple and reliable tool is needed. It should allow users to record expenses easily, check that the information entered is correct, save the data in a familiar and portable format, show how much of the budget is left, and provide spending summaries by category. The data should also be easy to export for further use or analysis.

Objectives

- Provide full creat, read, update and delete operations for expenses.
- Allow expenses to be grouped by category and summarized per category.
- Let the user set a budget and see budget, spent and remaining amounts with an overspend warning.
- Export all expenses to CSV for use in other tools.
- Reject invalid input and recover gracefully from storage errors instead of crashing.
- Record every action, warning and error in a log file for traceability.
- Keep the design modular and verify it with automated tests.

Scope and target users

Scope: single-user, offline, command-line application. Multi-user access, a graphical interface, cloud sync and per-month budgets are outside the current scope. **Target users:** college students managing a monthly allowance and individuals who want a simple, private expense log kept in Excel.

4. Functional Requirements

The system is divided into four major functional modules (expense management, category management, budget tracking, reporting) supported by shared validation, storage and logging components.

SI/No	Module	Requirement
1	Expense	System: Add an expense with amount, category and optional description; id and date are assigned automatically.
2	Expense	System: list all expenses, newest first.
3	Expense	System: Updates the amount, category and/or description of an expense identified by id; blank fields keep their current value.
4	Expense	System: delete an expense identified by id.
5	Category	System: add a category and reject blank or duplicate names.
6	Category	System: list categories in alphabetical order.
7	Budget	System: store one budget amount; setting it again replaces the previous value.
8	Budget	System: display budget, total spent and remaining balance, and warn when remaining is negative.
9	Report	System: show total spending per category, highest first.

10	Report	System: export all expenses to Expense_Report.csv with a header row.
11	Validation	System: reject non-numeric, zero or negative amounts, blank required text and invalid ids, showing a clear message.
12	Storage	System: create the workbook and its three sheets automatically if they do not exist.
13	Logging	System: log successful operations, rejected inputs and errors with timestamps.

Input / output structure

Operation	Input	Output
Add expense	amount, category, description	'Expense saved.' or an error message
View expenses	-	Rows (id, amount, category, description, date), newest first
Update expense	id; optional amount / category / description	'Expense N updated.' / 'Nothing to update.' / error
Delete expense	id	'Expense N deleted.' or error
Add / view category	name / -	'Category added' or duplicate / sorted list
Set / check budget	amount / -	'Budget saved.' / budget, spent, remaining
Category report	-	Category : Rs.total, sorted descending
Export CSV	-	Expense_Report.csv and its path

User workflow

On start, the program verifies or creates the workbook and shows the menu. The user enters a number (1-11); the program collects the needed inputs, validates them, performs the operation on the workbook, prints the result and returns to the menu. Invalid menu choices print 'Invalid choice.' and any unexpected error is caught, logged and reported without ending the program. Choosing 11 exits.

5. Non-functional Requirements

Sl/No	Category	Requirement	How it is met
-------	----------	-------------	---------------

1	Reliability	Bad input or storage problems must not crash the program or corrupt data.	Validators reject bad input before any write; OSError is wrapped in StorageError; main loop catches unexpected exceptions.
2	Usability	A first-time user should operate the tool without documentation.	Numbered menu, plain prompts, clear messages such as 'Amount must be a valid number.'
3	Maintainability	Each concern lives in one module; changes stay local.	Nine focused modules, layered design, storage details hidden in database.py, small functions with docstring-level clarity.
4	Error-handling strategy	Errors are classified and reported consistently.	ValidationError for user input, StorageError for file problems; functions return 'Error: ...' strings, never raw tracebacks to the user.
5	Logging / monitoring	Actions and failures must be traceable.	logger.py: all events (INFO+) to expense_tracker.log; warnings and errors also to the console.
6	Testability	Logic must be testable without touching real data.	Every function accepts an optional path so tests run on temporary workbooks; 52 unit tests.
7	Security / data safety	Data stays local; input cannot execute code.	The program works completely offline and does not use network connections or unsafe commands. User input is checked and cleaned before it is saved. The actual data files are also kept out of Git to help protect the user's personal information.
8	Performance / resource efficiency	Operations feel instant for personal scale data.	A single workbook load and save per operation, the full test suite runs in under one second.
9	Portability	Runs on Windows, macOS and Linux.	Only Python and openpyxl are required paths built with os.path.

6. System Architecture

The application follows a simple three-layer architecture. The **presentation layer** (main.py, utils.py) shows the menu and reads input. The **business-logic layer** (expense.py, categories.py, budget.py, report.py, validators.py) implements features and rules. The **data-access layer**

(database.py) is the only code that talks to openpyxl and the workbook. logger.py is a cross-cutting service used by all layers.

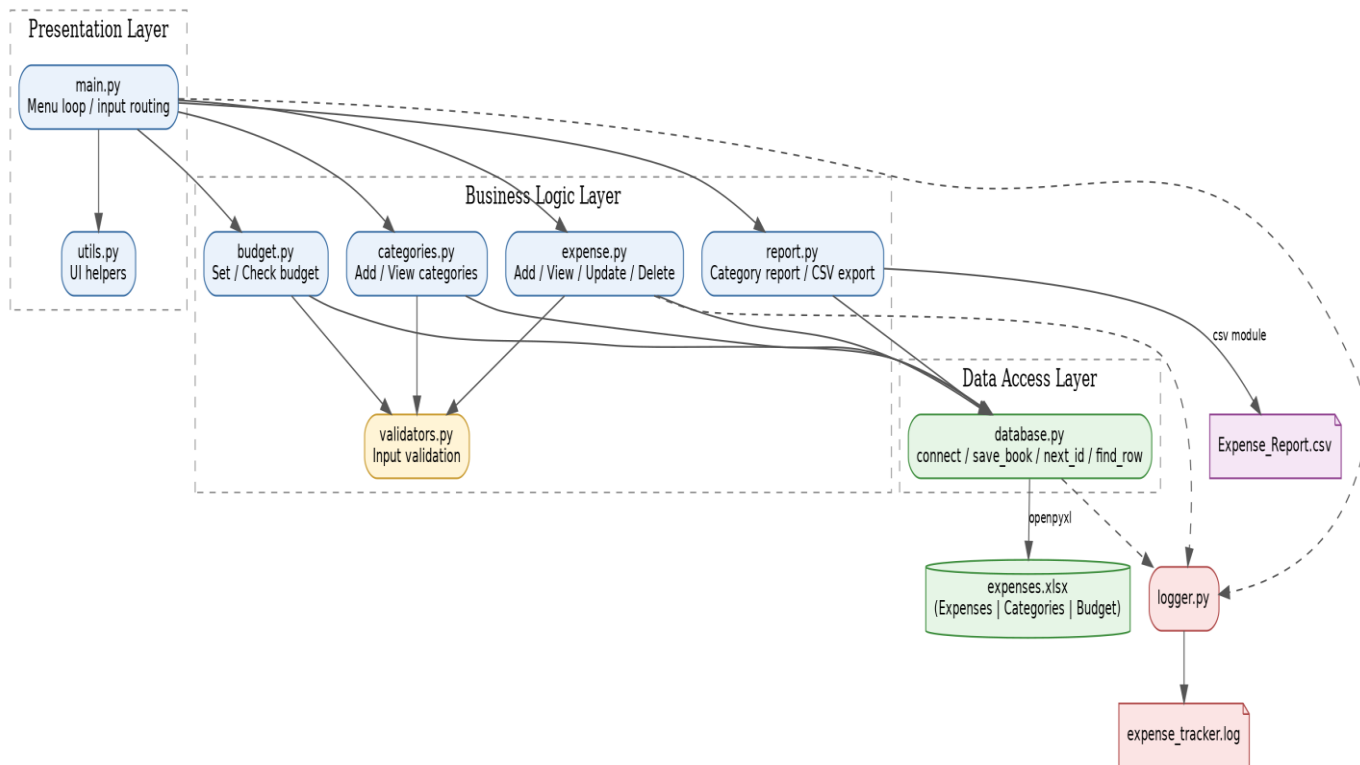


Figure 1: System architecture diagram

Layer responsibilities

Layer	Files	Responsibility
Presentation	main.py, utils.py	Menu loop, prompts, routing, top-level exception guard
Business logic	expense.py, categories.py, budget.py, report.py, validators.py	Feature logic, validation, result messages, logging of business events
Data access	database.py	Create/open/save workbook, read data rows, generate ids, find rows, wrap OS errors as StorageError
Cross-cutting	logger.py	File and console logging configuration

7. Design Diagrams

7.1 Use Case Diagram

The system has one main actor: the User. When adding or updating expenses, setting a budget, or creating a category, the program checks the entered information to make sure it is valid. Adding expenses and exporting data are also recorded in the activity log.

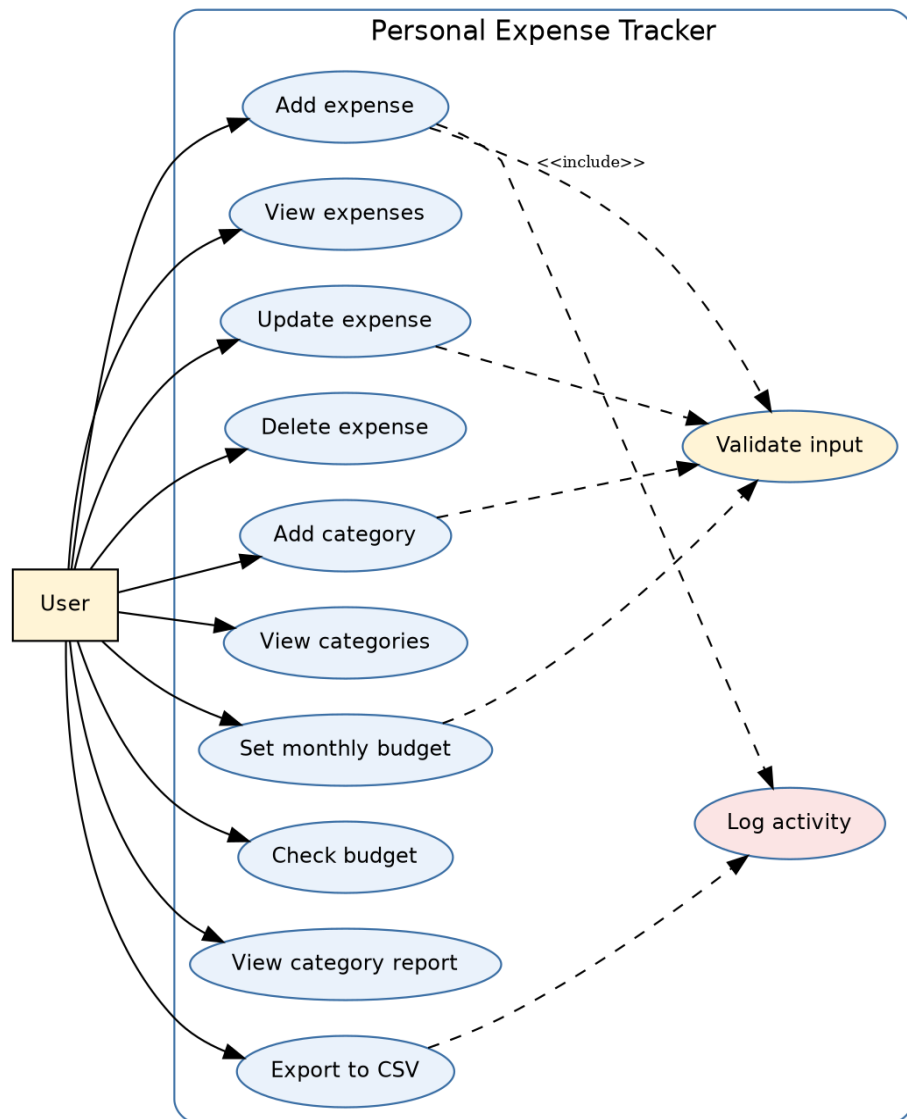


Figure 2: Use case diagram

7.2 Workflow (Process Flow) Diagram

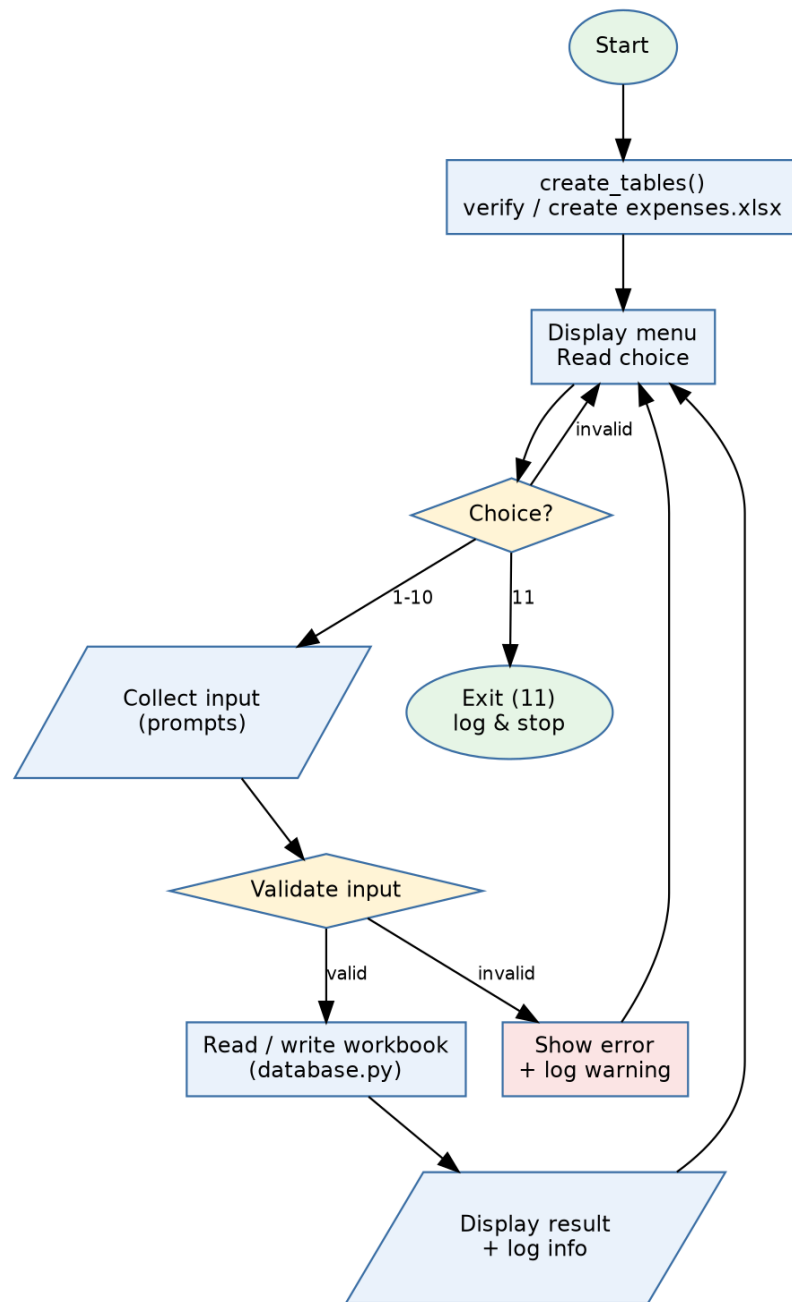


Figure 3: Workflow diagram

7.3 Sequence Diagram - Add Expense

The sequence shows how an expense is added step by step. The menu first calls `add_expense`, which takes the details from the user and checks that the entered values are valid. It then gets the workbook and the next available ID from `database.py`, adds the new expense, and saves the changes. Finally, the action is recorded in the log and a confirmation message is shown to the user.

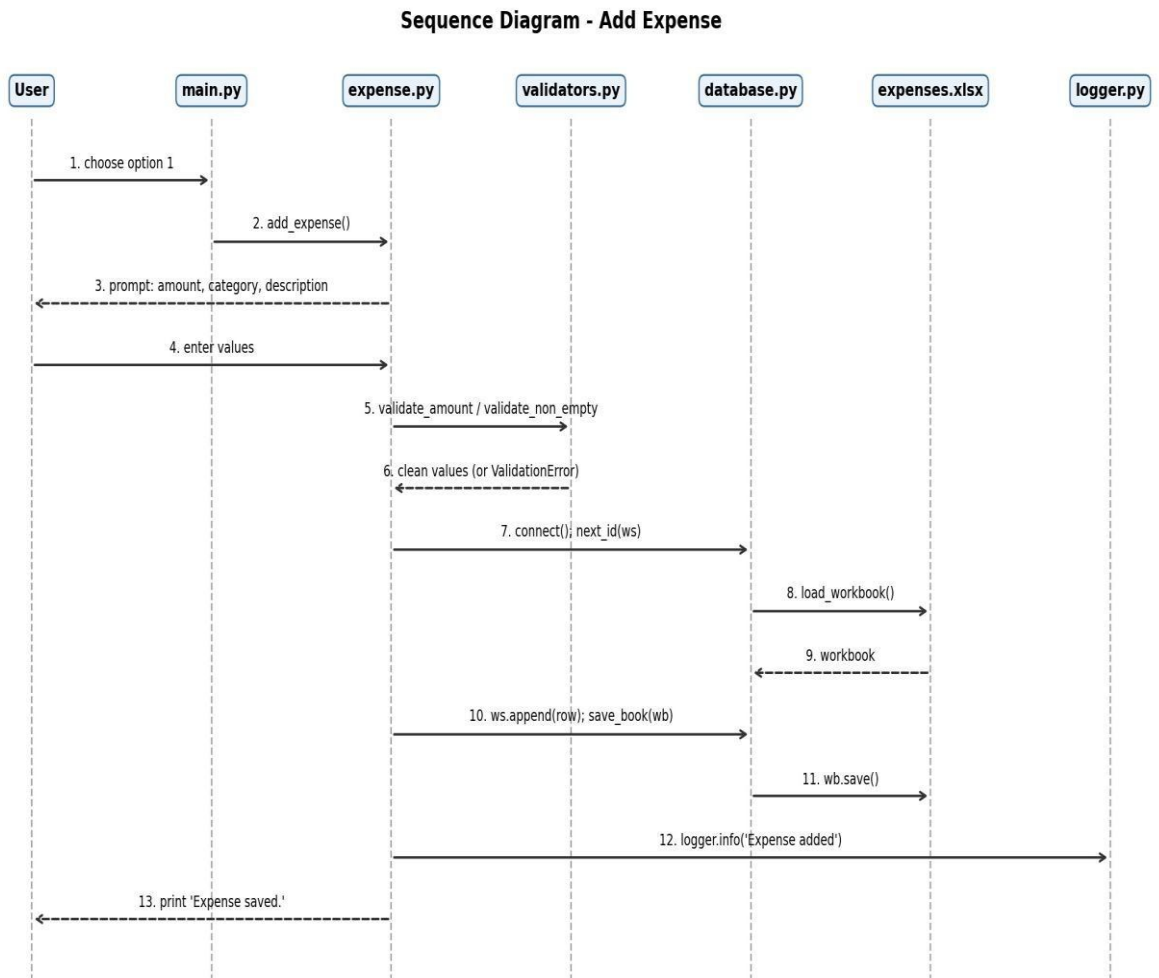


Figure 4: Sequence diagram for the Add Expense use case

7.4 Class / Component Diagram

The project is mainly based on functions rather than classes, so the diagram is shown as a component diagram. Each module is treated as a separate component and shows the main functions it provides. Solid arrows represent the connections between modules, while dashed arrows show where logging is used. The project also has two custom exceptions:

`ValidationError` in `validators.py` for invalid input and `StorageError` in `database.py` for storage-related problems.

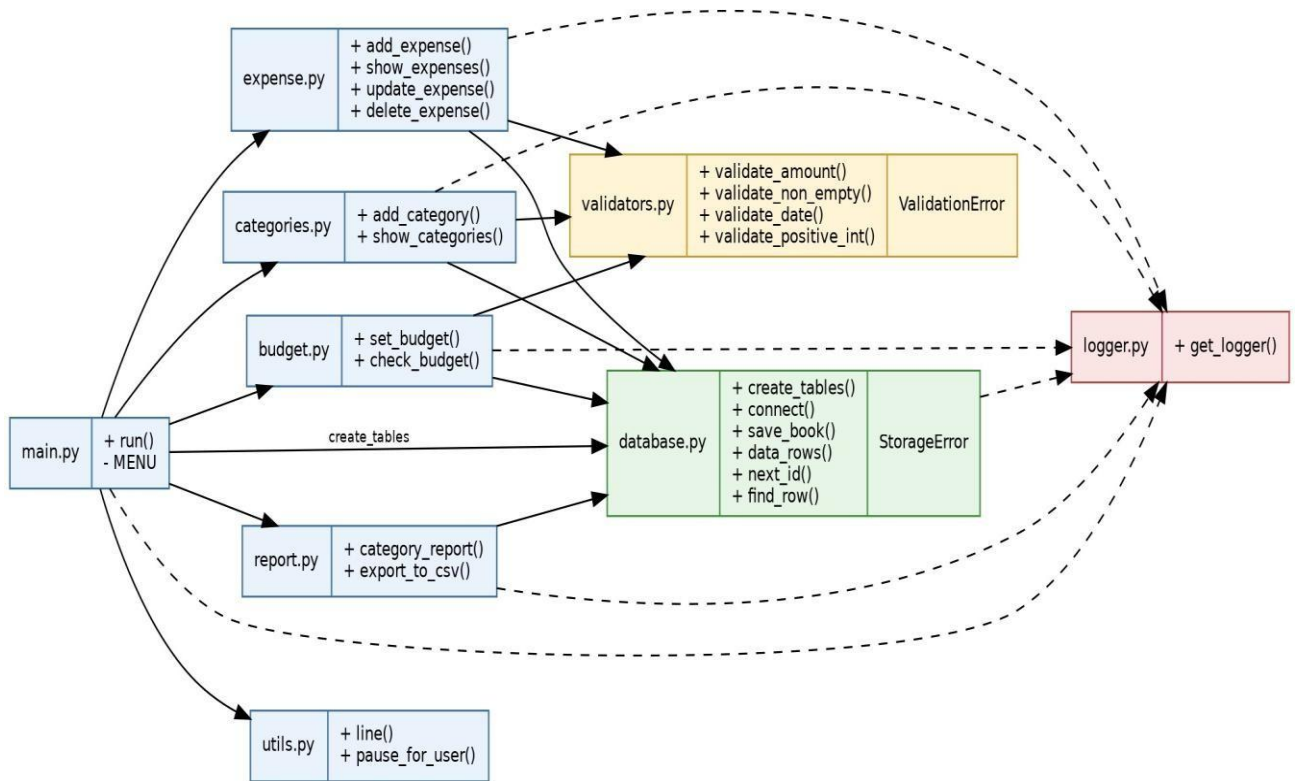


Figure 5: Component diagram

7.5 ER Diagram and Schema Design

Storage is a workbook with one sheet per entity. Row 1 of each sheet holds column headers; data starts on row 2. The link between Expenses and Categories is logical (by name) and is not enforced in the current version.

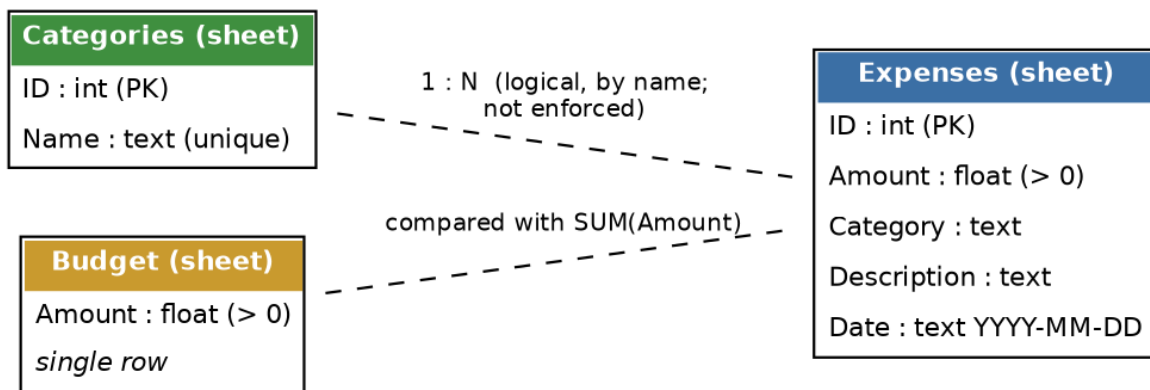


Figure 6: ER diagram of the workbook

Sheet	Column	Type	Rules
Expenses	ID	Integer	Primary key, max(existing)+1
Expenses	Amount	Float	> 0, rounded to 2 decimals
Expenses	Category	Text	Required, trimmed
Expenses	Description	Text	Optional

Expenses	Date	Text (YYYY-MM-DD)	Set automatically to today
Categories	ID	Integer	Primary key
Categories	Name	Text	Required, unique
Budget	Amount	Float	> 0; only one data row

8. Design Decisions & Rationale

Decision	Rationale	Trade-off
Excel workbook as storage instead of a database server	The project requires no complicated setup, works completely offline, and allows the data to be easily opened and edited in Excel. It is also simple to understand and easy to demonstrate.	The whole file is loaded and saved on every operation, and it cannot be edited in Excel while the program is saving. Suitable for personal-scale data only.
Layered modules with a dedicated dataaccess module	The main features of the project are kept separate from openpyxl, so the storage system can be changed easily. For example, if we wanted to switch from Excel to SQLite, we would only need to update the database.py file.	Slightly more files and indirection.
Separate validators.py that raises ValidationError	One place defines what valid input is; the same rules are reused by add, update and budget.	-
Custom StorageError wrapping OSError	The program uses a single type of exception, making it easier for users to understand errors and display helpful messages, such as asking them to close the Excel file if it is currently open.	-
Optional path argument on every function	Tests can run against a temporary workbook, so real data is never at risk.	Extra parameter on each function.

Central logger with file (INFO) and console (WARNING) handlers	The system records all activities in the file, while keeping the console simple and only showing messages when an error occur.	Log file grows without rotation.
Functions return result strings / data as well as printing	Return values make functions testable; printing keeps the CLI friendly.	Some functions do both.
Function-based modules instead of classes	The domain is small; plain functions are simpler to read, test and explain.	A class-based model (Expense, Category) would suit larger growth.
Amounts rounded to 2 decimals; dates stored as ISO text	Avoids floating-point noise and makes dates sortable and unambiguous.	Not real Excel date cells.

9. Implementation Details

9.1 Module overview

Module	Key functions	Notes
main.py	run()	Creates tables, logs start, loops on the menu, routes choices 1-11, catches unexpected exceptions and logs them.
expense.py	add_expense, show_expenses, update_expense, delete_expense	Uses validators for input, database helpers for storage, stamps today's date, builds an updates dictionary keyed by column number.
categories.py	add_category, show_categories	Rejects blank and duplicate names; lists names sorted.
budget.py	set_budget, check_budget	Replaces the single budget row; computes spent = sum of all expense amounts; prints a warning if remaining < 0.
report.py	category_report, export_to_csv	Aggregates totals in a dictionary and sorts descending; writes CSV with csv.writer and handles OSError.

database.py	create_tables, connect, save_book, data_rows, next_id, find_row	Only module using openpyxl; defines SHEETS schema and StorageError.
validators.py	validate_amount, validate_non_empty, validate_date, validate_positive_int	Return cleaned values or raise ValidationError.
logger.py	get_logger	Idempotent logger factory; file handler (INFO) and console handler (WARNING).
utils.py	line, pause_for_user	Console helpers.

9.2 Selected code

Validation (validators.py) - converts and range-checks an amount:

```
def validate_amount(value) -> float:    try:

    amount = float(value)
except (TypeError, ValueError):
    raise ValidationError(
        "Amount must be a valid number (e.g. 250.50)."
    )

if amount <= 0:
    raise ValidationError(
        "Amount must be greater than zero."
    )

return round(amount, 2)
```

Storage error wrapping (database.py) - a locked file becomes a friendly message:

```
def save_book(wb, path=None):    path = path or DB_NAME

try:
    wb.save(path)

except OSError as exc:
    logger.error(
        "Failed to save workbook %s: %s",
path,        exc
    )
```

```

        raise StorageError(
            "Could not write to the Excel file. "
            "Close it if it is open in Excel."
        ) from exc

Id generation and lookup (database.py): def
next_id(ws) -> int:
    return max(
        (row[0] for row in data_rows(ws)),
        default=0
    ) + 1

```

```

def find_row(ws, record_id: int):
    for row in ws.iter_rows(min_row=2):
        if row[0].value == record_id:
            return row[0].row

```

```

    return None

```

9.3 Subject concepts applied

- **Modularity and abstraction** - storage hidden behind database.py.
- **Exception handling** - custom exceptions, try/except at module and top-level boundaries.
- **Data structures** - lists of tuples for rows, dictionaries for category totals and pending updates, sorting with key functions.
- **File handling** - Excel via openpyxl, CSV via the csv module, log file via logging.
- **Testing** - unit tests with isolated temporary storage.

10. Screenshots / Results

The screenshots below were produced by running the application with scripted input; the text is the program's actual console output.

```
PERSONAL EXPENSE TRACKER
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit
Enter choice: |
```

Figure 7: Main menu

```
PERSONAL EXPENSE TRACKER
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit
Enter choice: 5
Enter category name: food
PERSONAL EXPENSE TRACKER
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit
Enter choice: 5
Enter category name: transport
PERSONAL EXPENSE TRACKER
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit
Enter choice: |
```

Figure 8: Adding categories

```
Enter choice: 1
Enter amount: 1000
Enter category: food
Description: dinner
Expense saved.
PERSONAL EXPENSE TRACKER
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit
Enter choice: 2
All Expenses
(1, 1000, 'food', 'dinner', '2026-09-28')
PERSONAL EXPENSE TRACKER
1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit
Enter choice: |
```

Figure 9: Expenses

```

Enter choice: 3
Expense ID to update: 1
New amount (leave blank to keep current): 400
New category (leave blank to keep current):
New description (leave blank to keep current):
Expense 1 updated.

```

PERSONAL EXPENSE TRACKER

1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit

```

Enter choice: 4
Expense ID to delete: 1
Expense 1 deleted.

```

PERSONAL EXPENSE TRACKER

1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit

Figure 10: Updating and deleting

```

Enter choice: 9

```

```

Expense by Category

```

```

Food : Rs.245.00

```

PERSONAL EXPENSE TRACKER

1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit

```

Enter choice: 10
Invalid choice.

```

PERSONAL EXPENSE TRACKER

1. Add Expense
2. View Expenses
3. Update Expense
4. Delete Expense
5. Add Category
6. View Categories
7. Set Budget
8. Check Budget
9. Category Report
10. Export to CSV
11. Exit

```

Enter choice: 10

```

```

Export successful!

```

```

File saved at:

```

```

C:\Users\user1\OneDrive\Desktop\expense_tracker\Expense_Report.csv

```

Figure 11: Budget

expense_tracker.log (real log from development runs)

```

2026-09-28 15:59:21.433 [INFO] database: Workbook verified/created at C:\Users\user1\OneDrive\Desktop\project\expenses.xlsx
2026-09-28 15:59:21.434 [INFO] _main : Application started.
2026-09-28 15:59:29.481 [INFO] categories: Category added: food
2026-09-28 15:59:30.779 [INFO] expense: Expense added: amount=1000.0 category=food
2026-09-28 15:59:59.520 [INFO] budget: Budget set to 20000.0
2026-09-28 16:00:07.557 [INFO] report: Exported 1 expenses to C:\Users\user1\OneDrive\Desktop\project\Expense_Report.csv
2026-09-28 16:02:26.508 [INFO] expense: Expense 1 updated.
2026-09-28 16:02:28.083 [INFO] report: Exported 1 expenses to C:\Users\user1\OneDrive\Desktop\project\Expense_Report.csv
2026-09-28 16:02:30.574 [INFO] _main : Application exited normally.
2026-09-28 18:43:56.619 [INFO] database: workbook verified/created at C:\Users\user1\OneDrive\Desktop\project\expenses.xlsx
2026-09-28 18:43:56.623 [INFO] _main : Application started.
2026-09-28 18:44:16.134 [INFO] expense: Expense added: amount=200.0 category=transport
2026-09-28 18:44:38.630 [INFO] categories: Category added: transport
2026-09-28 18:44:53.586 [ERROR] report: Failed to write export file C:\Windows\System32\Expense_Report.csv: [Error 13] Permission denied: 'C:\Windows\System32\Expense_Report.csv'
2026-09-28 18:44:56.445 [INFO] _main : Application exited normally.
2026-09-28 18:46:14.978 [INFO] database: workbook verified/created at C:\Users\user1\OneDrive\Desktop\project\expenses.xlsx
2026-09-28 18:46:14.979 [INFO] _main : Application started.
2026-09-28 18:46:24.285 [INFO] expense: Expense 1 deleted

```

Figure 12: Extract from expense_tracker.log recorded during development

11. Testing Approach

Testing combines automated unit tests and manual scenario checks. The automated suite contains 52 unit tests written with Python's built-in unittest, so

no extra installation is needed. Each test creates a fresh temporary workbook (base class `WorkbookTestCase`) and passes its path to the function under test, so the user's real data is never modified. Console output is captured to assert on messages.

```
python -m unittest discover -s tests -t . -v
```

Test file	Tests	What is verified
test_budget.py	5	Budget and all related to it are verified.
test_categories.py	6	Categories: add, duplicate, blank, trimming, sorted and empty listing
test_database.py	6	Database: table creation, idempotency, missing-file creation, ids, row lookup, blank rows
test_expense.py	18	Expense: valid/invalid add, id increment, ordering, update variants, delete variants
test_report.py	6	Report: empty report, grouping and sorting, CSV content, empty export, default export folder, unwritable folder
test_validators.py	11	Validators: amount, non-empty, date, positive integer - valid and invalid cases
Total	52	All tests pass

Test techniques

- **Boundary and negative testing** – The program is tested with different types of invalid input, such as zero, negative numbers, text instead of numbers, blank values, and missing or invalid IDs.
- **Isolation** – Each test uses its own temporary file, which is automatically removed after the test. This keeps the tests separate and protects the actual data.
- **Error handling** – The program is also tested with situations such as trying to export data to a folder that does not exist. This checks whether the file error is handled properly and a useful message is shown.
- **Regression testing** – The complete test suite takes less than a second to run, so it can easily be run again after making changes to make sure existing features still work correctly.

```

$ python -m unittest discover -s tests -t . -v
test_budget.BudgetTests
test_check_without_budget ok
test_invalid_budget_rejected ok
test_overspend_warning ok
test_set_and_check ok
test_set_budget_replaces_previous_value ok
test_categories.CategoryTests
test_add_category ok
test_blank_rejected ok
test_duplicate_rejected ok
test_name_is_trimmed ok
test_show_categories_empty ok
test_show_categories_sorted ok
test_database.DatabaseTests
test_connect_creates_missing_workbook ok
test_create_tables_builds_all_sheets_with_headers ok
test_create_tables_is_idempotent ok
test_data_rows_skips_header_and_blank_ids ok
test_next_id_and_find_row ok
test_next_id_on_empty_sheet_is_one ok
test_expense.AddExpenseTests
test_add_valid_expense ok
test_description_is_optional ok
test_id_increment ok
test_rejects_empty_category ok
test_rejects_invalid_amount ok
test_rejects_negative_amount ok
test_expense.DeleteExpenseTests
test_delete_existing ok
test_delete_invalid_id ok
test_delete_missing ok
test_delete_only_target_row ok
test_expense.ShowExpenseTests
test_empty_message ok
test_newest_first ok
test_expense.UpdateExpenseTests
test_invalid_amount_leaves_row_unchanged ok
test_invalid_id ok
test_missing_id ok
test_nothing_to_update ok
test_update_all_fields ok
test_update_amount_only ok
test_report.CategoryReportTests
test_empty ok
test_totals_grouped_and_sorted_desc ok
test_report.ExportTests
test_default_export_folder_is_project_folder_not_cwd ok
test_export_to_unwritable_location_is_handled ok
test_export_writes_csv ok
test_nothing_to_export ok
test_validators.ValidateAmountTests
test_accepts_number_and_numeric_string ok
test_rejects_non_numeric ok
test_rejects_zero_and_negative ok
test_rounds_to_two_decimals ok
test_validators.ValidateDateTests
test_invalid_dates ok
test_valid_date ok
test_validators.ValidateNonEmptyTests
test_error_message_uses_field_name ok
test_rejects_blank_and_none ok
test_strips_whitespace ok
test_validators.ValidatePositiveIntTests
test_invalid ok
test_valid ok
.....
Ran 52 tests in 0.681s

OK

```

Figure 13: Unit test run - all tests pass

Manual test scenarios

#	Scenario	Expected	Result
1	Add expense with amount 'abc'	Error message, nothing saved	Pass
2	Add category 'food' twice	Second attempt rejected, logged as warning	Pass
3	Update expense, leave fields blank	Blank fields keep current values	Pass
4	Set budget 0	Rejected: must be greater than zero	Pass (seen in log)
5	Export CSV when launched from a protected folder	File saved in project folder; if a folder is unwritable, error shown and program continues	Pass

6	Delete a non-existent id	'no expense found' message	Pass
---	--------------------------	----------------------------	------

12. Challenges Faced

Challenge	How it was addressed
Choosing storage that is simple yet persistent	Used an Excel workbook through openpyxl instead of a database server, keeping the project portable and the data easy to inspect.
Workbook locked while open in Excel	save_book() catches OSError and raises StorageError with an instruction to close the file.
Export failing with 'Permission denied'	While testing the program, CSV export sometimes failed with a “Permission denied” error. The log showed that the program was trying to save the CSV file inside C:\Windows\System32. This happened because the program used the current working directory, which could be System32 when the application was started through a shortcut. To fix this, the program was updated in two ways. First, OSError is now handled properly, so the error is logged and a clear message is shown instead of the program crashing. Second, the default export location was changed to the project folder, where expenses.xlsx is stored, instead of using the current working directory. A unit test was also added to make sure this problem does not happen
Keeping logic testable	Introduced the optional path argument, separated validators, and made functions return values as well as print.
Reliable ids after deletions	ids are generated as max(existing)+1 and rows are found by id rather than position.
Consistent error messages	Centralised validation in validators.py with clear messages and a single ValidationError type.
Noisy console during normal use	Logger writes everything to the file but only warnings and errors to the console.

Blank rows / header
handling in
Excel

`data_rows()` starts at row 2 and ignores rows with no id.

Known limitations found during testing

- **Budget calculation:** The budget is currently compared with the total spending from all expenses. It does not yet calculate spending for a specific month.
- **Expense categories:** Categories are entered as free text, so the program does not currently check whether the category already exists in the Categories sheet.
- **Add Category message:** When a duplicate category is added, the menu does not display the message returned by the Add Category function. As a result, the duplicate is mainly noticed through the log or warning shown in the console.

13. Learnings & Key Takeaways

- Keeping the user interface, program logic, and data storage separate makes the code easier to understand, test, and update.
- Checking user input early and using clear custom errors makes it easier to handle problems in a consistent way.
- Logging is very useful when finding and fixing errors. For example, the log helped us quickly find the permission problem with the `System32` folder.
- Allowing functions to accept a file path makes testing much easier because we can use temporary files instead of the user's actual data.
- Writing tests can reveal problems that may not be obvious at first. For example, Excel may store `100.0` as `100`, which can affect how the value appears when exported to CSV.
- Good documentation, such as requirements, diagrams, and a README, is an important part of the project and should not be left until the end.
- Using Git along with a proper `.gitignore` file helps prevent personal data files and log files from being accidentally added to the project repository.

14. Future Enhancements

- * Add monthly budgets and allow users to filter expenses by month or date.
- * Check expense categories against the Categories sheet and allow users to rename or delete categories.
- * Add simple charts, such as pie charts and bar charts, to make spending easier to understand.
- * Add search and filter options based on date, category, and amount, along with different sorting options.
- * Show a clear confirmation message when a new category is successfully added.
- * Add support for recurring expenses, reminders, and different currencies.
- * Use SQLite instead of Excel if the amount of data becomes larger, while keeping the existing ``database.py`` structure.

- * Create a more user-friendly interface using Tkinter or a web interface, with optional user accounts and password protection.
- * Add automatic log rotation and an option inside the program to view the log file.

15. References

Graphviz. (n.d.). Graphviz documentation. <https://graphviz.org/>

Matplotlib. (n.d.). Matplotlib documentation. <https://matplotlib.org/>

OpenPyXL. (n.d.). OpenPyXL documentation. <https://openpyxl.readthedocs.io/>

Python Software Foundation. (n.d.). CSV — CSV File Reading and Writing. Python documentation. <https://docs.python.org/3/library/csv.html>

Python Software Foundation. (n.d.). Logging — Logging facility for Python. Python documentation. <https://docs.python.org/3/library/logging.html>

Python Software Foundation. (n.d.). unittest — Unit testing framework. Python documentation. <https://docs.python.org/3/library/unittest.html>

Python Software Foundation. (n.d.). PEP 8 – Style Guide for Python Code. Python Enhancement Proposals. <https://peps.python.org/pep-0008/>

VITyarthi. (n.d.). Build Your Own Project guidelines [Course submission document].